CS1004 – Object Oriented Programming

Lecture # 24
Monday, May 16, 2022
Spring 2022
FAST – NUCES, Faisalabad Campus

Muhammad Yousaf

Today's Lecture

- Polymorphism in C++
- Pointers to derived classes
- Introduction to virtual functions

Review — Accessing Members of Base and Derived Classes

- Access to members of a class object is determined by the type of the handle.

- Definition: Handle
  - The thing by which the members of an object are accessed
  - May be
    - An object name (i.e., variable, etc.)
    - A reference to an object
    - A pointer to an object

Review — Accessing Members of Base and Derived Classes (continued)

- This is referred to as static binding
- i.e., the binding between handles and members is determined at compile time
  - i.e., statically

What if we need Dynamic Binding?

- What if we need a class in which access to methods is determined at run time by the type of the object, not the type of the handle

```
class Shape {
public:
void Rotate();
void Draw();
...
}
```

Need to access the method to draw the right kind of object Regardless of handle!

```
class Rectangle : public Shape {
public:
void Rotate();
void Draw();
...
}
```

```
class Ellipse : public Shape {
public:
void Rotate();
void Draw();
...
}
```

Solution – Virtual Functions

- Define a method as virtual, and the subclass method overrides the base class method

- For example

```
class Shape {
public:
virtual void Rotate();
virtual void Draw();
...
}
```

This tells the compiler to add internal pointers to every object of class Shape and its derived classes, so that pointers to correct methods can be stored with each object.

What if we need Dynamic Binding?

```
class Shape {
public:
virtual void Rotate();
virtual void Draw();
...
}
```

- Subclass methods override the base class methods
  - (if specified)
- C++ dynamically chooses the correct method for the class from which the object was instantiated

```
class Rectangle : public Shape {
public:
void Rotate();
void Draw();
...
}
```

```
class Ellipse : public Shape {
public:
void Rotate();
void Draw();
...
}
```

Notes on virtual

- If a method is declared virtual in a class,
  - … it is automatically virtual in all derived classes
- It is a really, really good idea to make destructors virtual!
  - virtual ~Shape();
  - Reason: to invoke the correct destructor, no matter how object is accessed

Virtual Destructors

- Constructors cannot be virtual, but destructors can be virtual

- It ensures that the derived class destructor is called when a base class pointer is used while deleting a dynamically created derived class object

Virtual Destructors (contd.)

```
#include <iostream>
using namespace std;

public:
~base() {
        cout << "destructing base\n";
}
};

class derived : public base {
public:
~derived() {
        cout << "destructing derived\n";
}
};
```

```
int main()
{
base *p = new derived;
delete p;
return 0;
}
```

```
Output:
destructing base
```

Using non-virtual destructor

Virtual Destructors (contd.)

```
class base {

public:
virtual ~base() {
        cout << "destructing base\n";
}
};

class derived : public base {
public:
~derived() {
        cout << "destructing derived\n";
}
};
```

```
int main()
{
base *p = new derived;
delete p;

return 0;
}
```

Output:

```
destructing derived
destructing base
```

Using virtual destructor

Notes on virtual (continued)

- A derived class may optionally override a virtual function
  - If not, base class method is used

```
class Shape {
public:
virtual void Rotate();
virtual void Draw();
...
}
class Line : public Shape {
public:
void Rotate(); //Use base class Draw method...
}
```

Summary – Based and Derived Class Pointers

- Base-class pointer pointing to base-class object
  - Straightforward
- Derived-class pointer pointing to derived-class object
  - Straightforward
- Base-class pointer pointing to derived-class object
  - Safe
  - **Can access non-virtual methods of only base-class**
  - **Can access virtual methods of derived class**
- Derived-class pointer pointing to base-class object
  - Compilation error

Abstract and Concrete Classes

- **Abstract Classes**
  - Classes from which it is never intended to instantiate any objects
    - Incomplete—derived classes must define the "missing pieces"
    - Too generic to define real objects
  - Normally used as base classes and called abstract base classes
    - Provide appropriate base class frameworks from which other classes can inherit
- **Concrete Classes**
  - Classes used to instantiate objects
    - Must provide implementation for every member function they define

Pure virtual Functions

- A class is made abstract by declaring one or more of its virtual functions to be "pure"
  - By placing "= 0" in its declaration

- Example

```
virtual void draw() const = 0;
```

  - " = 0" is known as a pure specifier
  - Tells compiler that there is no implementation

Pure `virtual` Functions (continued)

- Every concrete derived class must override all base-class pure virtual functions
  - with concrete implementations
- If even one pure virtual function is not overridden, the derived-class will also be abstract
  - Compiler will refuse to create any objects of the class
  - Cannot call a constructor

Purpose

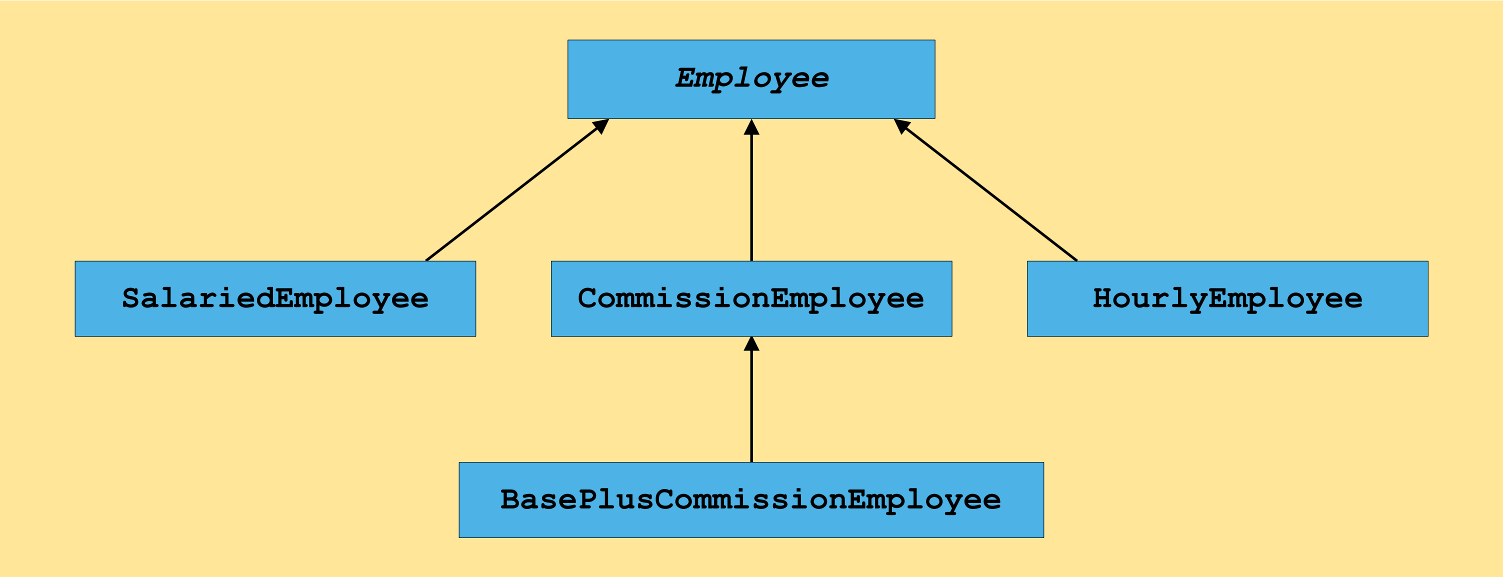
- When it does not make sense for base class to have an implementation of a function
- Software design requires all concrete derived classes to implement the function
  - Themselves

Why do we want to do this?

- To define a common public *interface* for various classes in a class hierarchy
  - Create framework for abstractions defined in our software system
- The heart of *object-oriented programming*
- Simplifies a lot of big software systems
  - Enables code re-use in a major way
  - Readable, maintainable, adaptable code

- Create a payroll program
  - Use virtual functions and polymorphism
- Problem statement
  - 4 types of employees, paid weekly
    - Salaried (fixed salary, no matter the hours)
    - Hourly workers
    - Commission (paid percentage of sales)
    - Base-plus-commission (base salary + percentage of sales)

- Base class Employee
  - Pure virtual function earnings (returns pay)
    - Pure virtual because need to know employee type
    - Cannot calculate for generic employee
  - Other classes derive from Employee



Employee Example

```
class Employee {
public:
Employee(const char *, const char *);
~Employee();
char *getFirstName() const;
char *getLastName() const;

// Pure virtual functions make Employee abstract base class.
virtual float earnings() const = 0; // pure virtual
virtual void print() const = 0;     // pure virtual

protected:
char *firstName;
char *lastName;
};
```

```
Employee::Employee(const char *first, const char *last)
{
firstName = new char[strlen(first) + 1];
strcpy(firstName, first);
lastName = new char[strlen(last) + 1];
strcpy(lastName, last);
}

// Destructor deallocates dynamically allocated memory
Employee::~Employee() {
delete[] firstName;
delete[] lastName;
}

// Return a pointer to the first name
char *Employee::getFirstName() const {
return firstName;   // caller must delete memory
}

char *Employee::getLastName() const {
return lastName;   // caller must delete memory
}
```

```
class SalariedEmployee : public Employee {
public:
SalariedEmployee(const char *, const char *, float = 0.0);
void setWeeklySalary(float);
virtual float earnings() const;
virtual void print() const;
private:
float weeklySalary;
};
```

```cpp
// Constructor function for class
SalariedEmployee::SalariedEmployee(const char *first,
const char *last, float s)
: Employee(first, last)  // call base-class constructor
{
weeklySalary = s > 0 ? s : 0;
}

// Set the SalariedEmployee’s salary
void SalariedEmployee::setWeeklySalary(float s)
{
weeklySalary = s > 0 ? s : 0;
}

// Get the SalariedEmployee’s pay
float SalariedEmployee::earnings() const { return weeklySalary; }

// Print the SalariedEmployee’s name
void SalariedEmployee::print() const
{
cout << endl << " Salaried Employee: " << getFirstName()
            << ' ' << getLastName();
}
```

```
class CommissionWorker : public Employee {
public:
CommissionWorker(const char *, const char *, float = 0.0, unsigned = 0);
void setCommission(float);
void setQuantity(unsigned);
virtual float earnings() const;
virtual void print() const;

private:
float commission;    // amount per item sold
unsigned quantity;   // total items sold for week
};
```

```
CommissionWorker::CommissionWorker(const char *first,
const char *last, float c, unsigned q): Employee(first, last)  // call base-class constructor
{
commission = c > 0 ? c : 0;
quantity = q > 0 ? q : 0;
}
void CommissionWorker::setCommission(float c){
commission = c > 0 ? c : 0;
}
void CommissionWorker::setQuantity(unsigned q){
quantity = q > 0 ? q : 0;
}
float CommissionWorker::earnings() const{
return commission * quantity;
}
void CommissionWorker::print() const{
cout << endl << "Commission worker: " << getFirstName()
         << ' ' << getLastName();
}
```

```
class HourlyWorker : public Employee {
public:
HourlyWorker(const char *, const char *, float = 0.0, float = 0.0);
void setWage(float);
void setHours(float);
virtual float earnings() const;
virtual void print() const;
private:
float wage;    // wage per hour
float hours;   // hours worked for week
};
```

```
HourlyWorker::HourlyWorker(const char *first, const char *last,
float w, float h) : Employee(first, last)   // call base-class constructor
{
wage = w > 0 ? w : 0;
hours = h >= 0 && h < 168 ? h : 0;
}
void HourlyWorker::setWage(float w) { wage = w > 0 ? w : 0; }
// Set the hours worked
void HourlyWorker::setHours(float h)
{
hours = h >= 0 && h < 168 ? h : 0;
}
// Get the HourlyWorker's pay
float HourlyWorker::earnings() const { return wage * hours; }
// Print the HourlyWorker's name
void HourlyWorker::print() const
{
cout << endl << "    Hourly worker: " << getFirstName()
<< ' ' << getLastName();
}
```

```
BasePlusCommissionEmployee:public CommissionWorker
{
private:
float baseSalary;
public:
BasePlusCommissionEmployee(const char* , const char* , float =0.0,
                                                                      unsigned =0,float
=0.0);
void setBaseSalary(float sal){
        baseSalary = sal;
}
float getBaseSalary(void) const {
        return baseSalary;
}
void print() const;
float earnings() const;
};
```

```
BasePlusCommissionEmployee::BasePlusCommissionEmployee(const char* first, const
char* last, float c, unsigned q, float sal) :CommissionWorker(first, last, c, q)
{
baseSalary = (sal);
}
void BasePlusCommissionEmployee::print() const
{
cout << "\nbase-salaried commission employee: ";
CommissionWorker::print();  // code reuse
} // end function print
float BasePlusCommissionEmployee::earnings() const
{
return getBaseSalary() + CommissionWorker::earnings();
} // end function earnings
```

```
int main(void)
{
Employee *ptr;  // base-class pointer

SalariedEmployee b("Nauman", "Sarwar", 800.00);
ptr = &b;  // base-class pointer to derived-class object
ptr->print();                               // dynamic binding
cout << " earned $" << ptr->earnings(); // dynamic binding
b.print();                                  // static binding
cout << " earned $" << b.earnings();    // static binding

CommissionWorker c("Qasim", “Ali", 3.0, 150);
ptr = &c;  // base-class pointer to derived-class object
ptr->print();                               // dynamic binding
cout << " earned $" << ptr->earnings(); // dynamic binding
c.print();                                  // static binding
cout << " earned $" << c.earnings();    // static binding
```

```
BasePlusCommissionEmployee p("Mehshan", "Mustafa", 2.5, 200, 1000.0);
ptr = &p;  // base-class pointer to derived-class object
ptr->print();                               // dynamic binding
cout << " earned $" << ptr->earnings(); // dynamic binding
p.print();                                  // static binding
cout << " earned $" << p.earnings();    // static binding

HourlyWorker h("Samer", "Tufail", 13.75, 40);
ptr = &h;  // base-class pointer to derived-class object
ptr->print();                               // dynamic binding
cout << " earned $" << ptr->earnings(); // dynamic binding
h.print();                                  // static binding
cout << " earned $" << h.earnings();    // static binding

cout << endl;

return 0;
}
```

Output

C:\Windows\system32\cmd.exe

```
Salaried Employee: Nauman Sarwar earned $800
Salaried Employee: Nauman Sarwar earned $800
Commission worker: Qasim Ali earned $450
Commission worker: Qasim Ali earned $450
base-salaried commission employee:
Commission worker: Mehshan Mustafa earned $1500
base-salaried commission employee:
Commission worker: Mehshan Mustafa earned $1500
Hourly worker: Samer Tufail earned $550
Hourly worker: Samer Tufail earned $550
Press any key to continue . . .
```

Questions